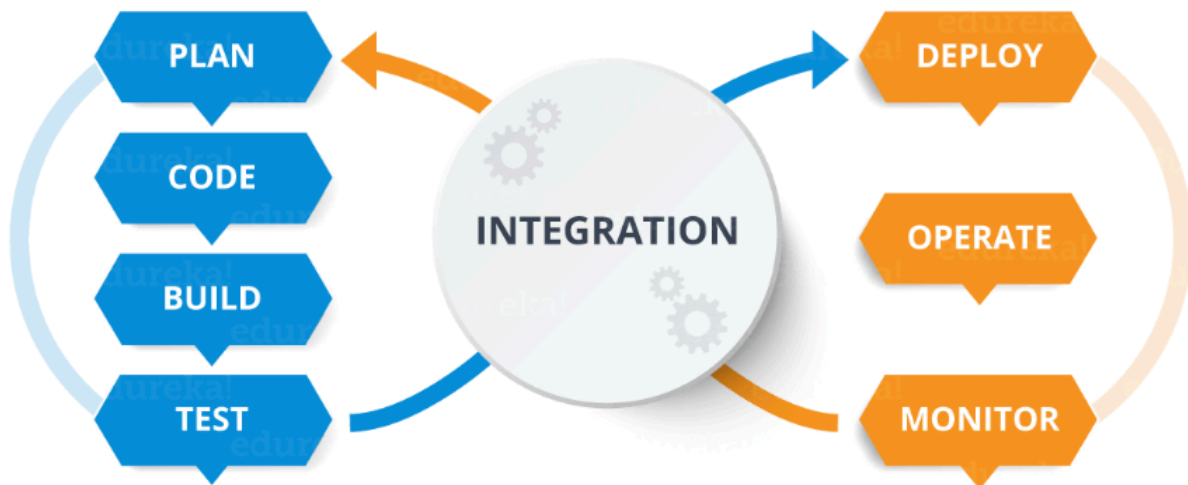


DevOps

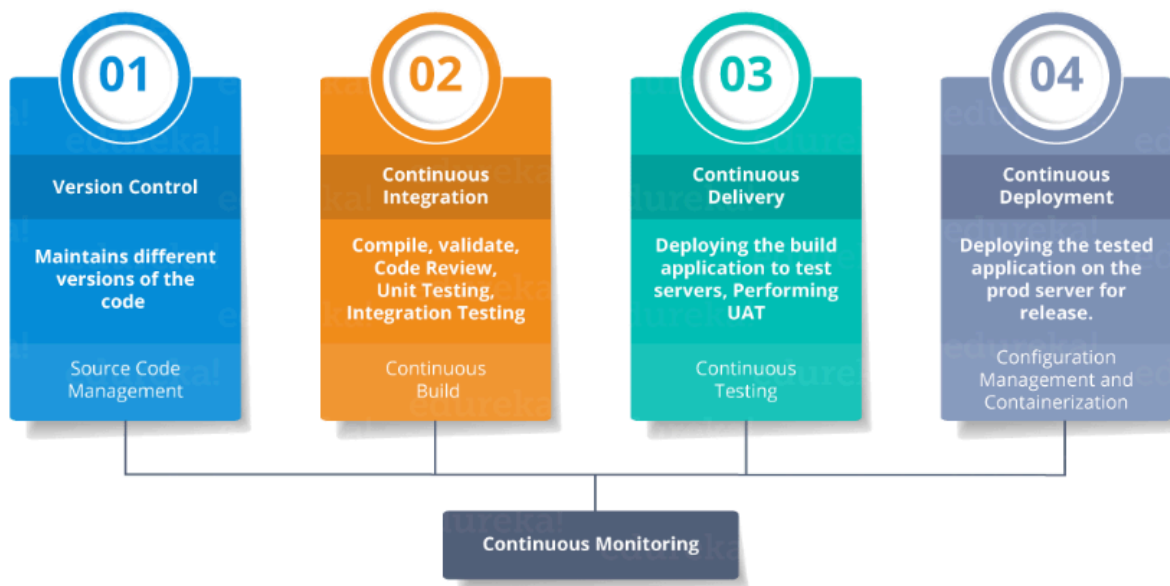
A CI/CD Pipeline implementation, or Continuous Integration/Continuous Deployment, is the backbone of the modern DevOps environment. It bridges the gap between development and operations teams by automating the building, testing, and deployment of applications. In this blog, we will learn what a CI/CD pipeline is and how it works.

Before moving onto the CI/CD pipeline, let's start by understanding DevOps.



DevOps is a software development approach which involves continuous development, continuous testing, continuous integration, continuous deployment, and continuous monitoring of the software throughout its development life cycle. This is the process adopted by all the top companies to develop high-quality software and shorter development life cycles, resulting in greater customer satisfaction, something that every company wants.

The understanding of DevOps is incomplete without learning about its life cycle. Let us now look at the DevOps life cycle and explore how it is related to the software development stages.



Jenkins

Jenkins is an open source automation server written in Java. Jenkins helps to automate the non-human part of the software development process, with continuous integration and facilitating technical aspects of continuous delivery. It is a server-based system that runs in servlet containers such as Apache Tomcat.

Docker

Docker is a collection of interoperating software-as-a-service and platform-as-a-service offerings that employ operating-system-level virtualization to cultivate development and delivery of software inside standardized software packages called containers. The software that hosts the containers is called Docker Engine.

Example Overview

So, our aim today is to create a CI/CD pipeline from a simple Spring Boot application. The application has a simple index.html page, and some Spring Boot annotations in order to be able to run, and to build this as a jar. Then, to build our project we are using Maven, thus let's delve into the commented pom.xml file.

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
```

```

        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>2.1.5.RELEASE</version>
        <relativePath/> <!-- lookup parent from repository -->
    </parent>
    <groupId>com.devops</groupId>
    <artifactId>devOpsDemo</artifactId>
    <version>0.0.1-SNAPSHOT</version>
    <name>devOpsDemo</name>
    <packaging>jar</packaging>
    <description>Demo project for Spring Boot</description>

    <properties>
        <java.version>1.8</java.version>
    </properties>

    <dependencies>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-rest</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-freemarker</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-thymeleaf</artifactId>
        </dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-devtools</artifactId>
            <scope>runtime</scope>
        </dependency>
        <dependency>

```

```

        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-compiler-plugin</artifactId>
            <version>3.7.0</version>
            <configuration>
                <source>1.8</source>
                <target>1.8</target>
                <jdkToolchain>
                    <version>8</version>
                </jdkToolchain>
            </configuration>
        </plugin>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-jar-plugin</artifactId>
            <version>3.0.2</version>
        </plugin>
    </plugins>
</build>

</project>

```

This sample was built using Spring Initializr, and we have here a bunch of dependencies that are not being used, like Freemarker and Thymeleaf. The important part here, is the maven build plugin, enabling us to run the mvn clean install command, and build our application as a jar.

The Spring Boot application, and all the files needed for this tutorial are in the github repository:

Dockerfile

First of all we have to build our image to be able to dockerize it, running the docker build command. Therefore, in the github repository we can check for the following Dockerfile.

```
FROM openjdk:8-jdk-alpine
```

```
VOLUME /tmp
```

```
ADD target/devOpsDemo-0.0.1-SNAPSHOT.jar app.jar
```

```
ENTRYPOINT ["java","-jar","app.jar"]
```

```
EXPOSE 2222
```

Basically, we are importing the available openjdk image from the docker official repositories, and adding our built jar file from the /target path. After that, we will specify the command to be executed after requesting the container to run, and exposing the 2222 port.

Under the project root, we can execute the three following commands:

- \$ mvn clean install - (Build the project, needs maven in the PATH environment variable)
- \$ docker build -t devopsexample . - (Build the docker image)
- \$ docker run --name devopsexample -p 2222:2222 -d devopsexample

In these three last steps, we have built our project files and generated jar and also our docker image. Then, we finally ran our application on the container.

We did it manually only for academic purposes, because the pipeline will do this automatically.

Running Jenkins

As we are using docker for running our application locally, nothing better to run the Jenkins also in a docker container, despite there is a big issue in this assumption, that can be solved with many workarounds. If we are running Jenkins server inside a docker container, how can we access the other containers, and also execute major docker commands? To solve this, I've created a Dockerfile that configures a Jenkins environment capable of running docker commands, and managing containers.

```
from jenkins/jenkins:lts
```

```
USER root
```

```
RUN apt-get update -qq \  
    && apt-get install -qqy apt-transport-https ca-certificates curl gnupg2  
software-properties-common
```

```
RUN curl -fsSL https://download.docker.com/linux/debian/gpg | apt-key add -
```

```
RUN add-apt-repository \  
"deb [arch=amd64] https://download.docker.com/linux/debian \  
$(lsb_release -cs) \  
stable"
```

```
RUN apt-get update -qq \  
    && apt-get install docker-ce=17.12.1~ce-0~debian -y
```

```
RUN usermod -aG docker jenkins
```

You just have to put this file in any directory, and execute the two following commands:

- `$ docker image build -t jenkins-docker .`
- `$ docker container run -d -p 8080:8080 -v /var/run/docker.sock:/var/run/docker.sock jenkins-docker`

After this, you will have a local Jenkins server running, we just have to configure it now.

Configuring Jenkins

Accessing Jenkins on localhost:8080, you will be requested to unlock Jenkins, and if after first running the container you haven't received the initial admin password, we will have to search for it inside the container.

The image shows the Jenkins 'Unlock Jenkins' screen. At the top, it says 'Getting Started'. The main heading is 'Unlock Jenkins'. Below it, a message states: 'To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:'. A code block shows the file path: `/var/jenkins_home/secrets/initialAdminPassword`. Below this, it says 'Please copy the password from either location and paste it below.' There is a label 'Administrator password' above a text input field. A 'Continue' button is at the bottom right. A faint background image of a person in a hard hat is visible on the right side.

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

```
/var/jenkins_home/secrets/initialAdminPassword
```

Please copy the password from either location and paste it below.

Administrator password

Continue

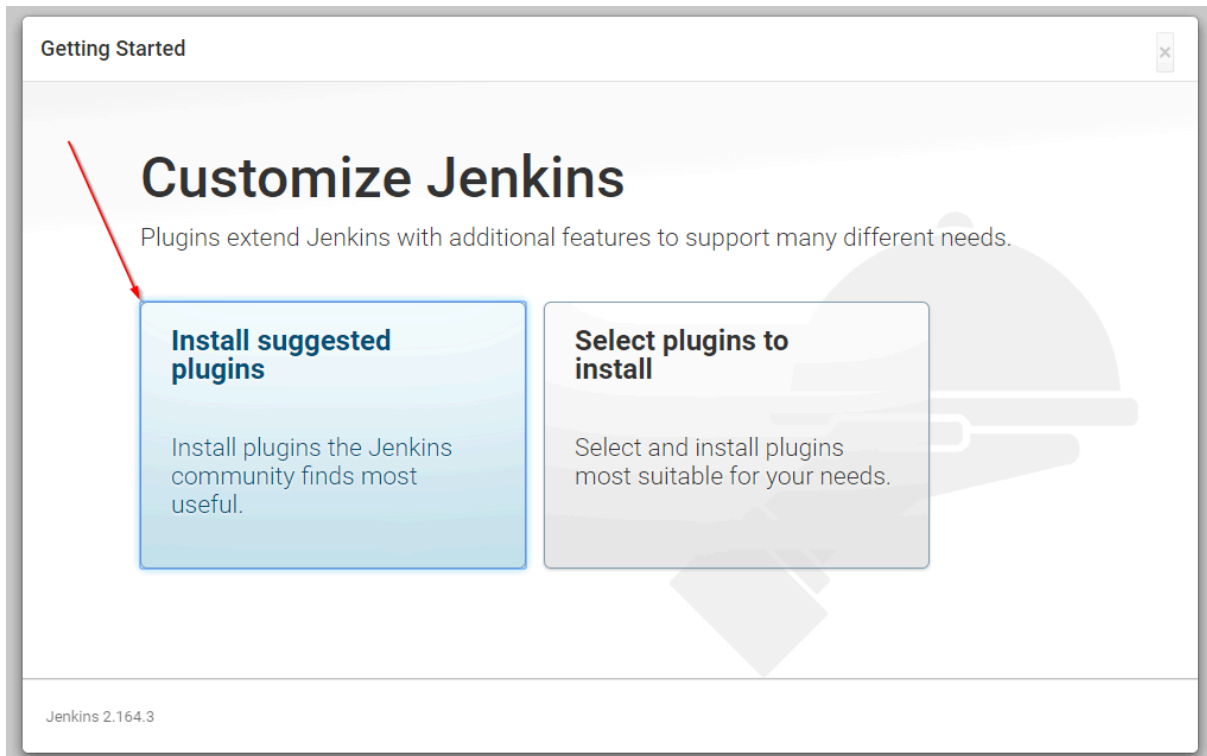
To access the container run the following command:

```
$ docker exec -it <your_container_name> /bin/bash
```

And get the initial admin password:

```
$ cat /var/jenkins_home/secrets/initialAdminPassword
```

Further, select install suggested plugins, and the Jenkins will download automatically the basic plugins.



Jenkins Global Configurations

First of all we will have to configure the JDK, Maven and GIT in the global configurations in order to enable Jenkins to build the application, and clone the code from the repository. So, in the Jenkins console, go to Manage Jenkins, and after that Global Tool Configuration.



-  New Item
-  People
-  Build History
-  Manage Jenkins
-  My Views
-  Credentials
-  Lockable Resources
-  New View

Build Queue 

No builds in the queue.

Build Executor Status 

1 Idle
2 Idle

Manage Jenkins

-  **Configure System**
Configure global settings and paths.
-  **Configure Global Security**
Secure Jenkins; define who is allowed to access/use the system.
-  **Configure Credentials**
Configure the credential providers and types
-  **Global Tool Configuration**
Configure tools, their locations and automatic installers.
-  **Reload Configuration from Disk**
Discard all the loaded data in memory and reload everything from file system. Useful when you modified config files directly on disk.
-  **Manage Plugins**
Add, remove, disable or enable plugins that can extend the functionality of Jenkins.

JDK

We will use the open jdk that comes embedded with the Jenkins container, and the JDK directory will be located in /usr/lib/jvm/java-8-openjdk-amd64. Then, point out the path of your JDK installation inside container,

JDK

JDK installations

JDK
NameJDK JAVA_HOME/usr/lib/jvm/java-8-openjdk-amd64 ☐ Install automatically

[Delete JDK](#)

[Add JDK](#)

[List of JDK installations on this system](#)

Maven

Instead of point a maven directory on the system, we will tell Jenkins to download the Maven from apache servers. Therefore, just set it like the image bellow:

Maven

Maven installations

Maven
Namemaven-3.5.2 ☒ Install automatically Install from Apache Version3.5.2

[Delete Installer](#)

[Add Installer](#)

The Pipeline

Now that everything is set up, we can move to the pipeline. In Jenkins, you have the option to write the pipeline in the console, or let jenkins to find it in our git repository. You just have a file called Jenkinsfile in the root path of the repository.

Jenkins Pipeline (or simply "Pipeline" with a capital "P") is a suite of plugins which supports implementing and integrating continuous delivery pipelines into Jenkins.

A continuous delivery (CD) pipeline is an automated expression of your process for getting software from version control right through to your users and customers. Every change to your software (committed in source control) goes through a complex process on its way to being released. This process involves building the software in a

reliable and repeatable manner, as well as progressing the built software (called a "build") through multiple stages of testing and deployment.

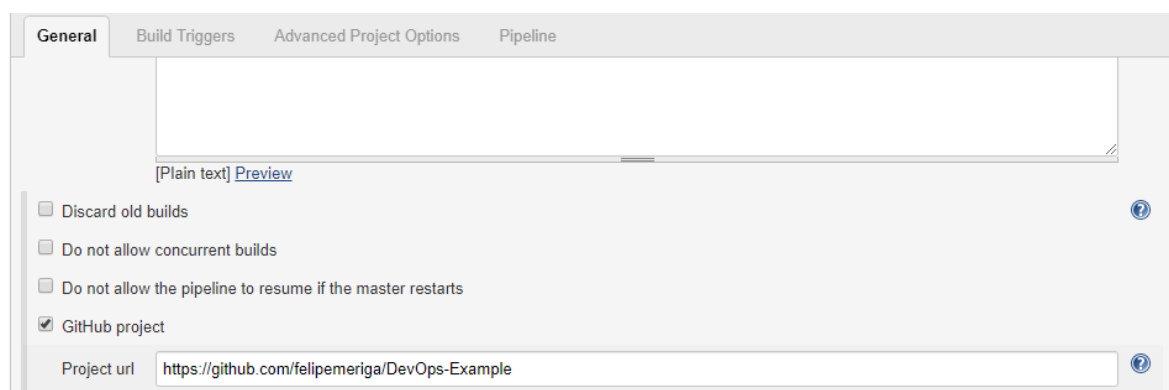
Pipeline provides an extensible set of tools for modeling simple-to-complex delivery pipelines "as code" via the Pipeline domain-specific language (DSL) syntax.

The definition of a Jenkins Pipeline is written into a text file (called a Jenkinsfile) which in turn can be committed to a project's source control repository. This is the foundation of "Pipeline-as-code"; treating the CD pipeline a part of the application to be versioned and reviewed like any other code.

Creating a Jenkinsfile and committing it to source control provides a number of immediate benefits:

- Automatically creates a Pipeline build process for all branches and pull requests.
- Code review/iteration on the Pipeline (along with the remaining source code).
- Audit trail for the Pipeline.
- Single source of truth for the Pipeline, which can be viewed and edited by multiple members of the project.

Then, let's create the pipeline from our git repository. Just hit new item, give it a name and select pipeline. You will reach the section to configure, your pipeline, you just have to inform Jenkins that your domain-specific-language file(Jenkinsfile) is under your git root directory.



The screenshot shows the Jenkins configuration interface for a new Pipeline. At the top, there are four tabs: 'General' (selected), 'Build Triggers', 'Advanced Project Options', and 'Pipeline'. Below the tabs is a large text area for the Pipeline script, currently empty, with a '[Plain text] Preview' link below it. On the left side, there are four checkboxes: 'Discard old builds' (unchecked), 'Do not allow concurrent builds' (unchecked), 'Do not allow the pipeline to resume if the master restarts' (unchecked), and 'GitHub project' (checked). At the bottom, there is a 'Project url' field with the value 'https://github.com/felipemeriga/DevOps-Example' and a help icon.

Pipeline

Definition Pipeline script from SCM

SCM Git

Repositories

Repository URL https://github.com/felipemeriga/DevOps-Exa

Credentials - none - Add

Advanced...

Add Repository

Branches to build

Branch Specifier (blank for 'any') */master

Add Branch

Repository browser (Auto)

Now let's check the pipeline code, and further test it.

```
node {
    // reference to maven
    // ** NOTE: This 'maven-3.5.2' Maven tool must be configured in the Jenkins Global Configuration.
    def mvnHome = tool 'maven-3.5.2'

    // holds reference to docker image
    def dockerImage
    // ip address of the docker private repository(nexus)

    def dockerImageTag = "devopsexample${env.BUILD_NUMBER}"

    stage('Clone Repo') { // for display purposes
        // Get some code from a GitHub repository
        git 'https://github.com/felipemeriga/DevOps-Example.git'
        // Get the Maven tool.
        // ** NOTE: This 'maven-3.5.2' Maven tool must be configured
        // ** in the global configuration.
        mvnHome = tool 'maven-3.5.2'
    }

    stage('Build Project') {
        // build project via maven
        sh "${mvnHome}/bin/mvn clean install"
    }
}
```

```

stage('Build Docker Image') {
    // build docker image
    dockerImage = docker.build("devopsexample:${env.BUILD_NUMBER}")
}

stage('Deploy Docker Image'){

    // deploy docker image to nexus

    echo "Docker Image Tag Name: ${dockerImageTag}"

    sh "docker stop devopsexample"

    sh "docker rm devopsexample"

    sh "docker run --name devopsexample -d -p 2222:2222
devopsexample:${env.BUILD_NUMBER}"

    // docker.withRegistry('https://registry.hub.docker.com', 'docker-hub-credentials') {
    //   dockerImage.push("${env.BUILD_NUMBER}")
    //   dockerImage.push("latest")
    // }

}

}A

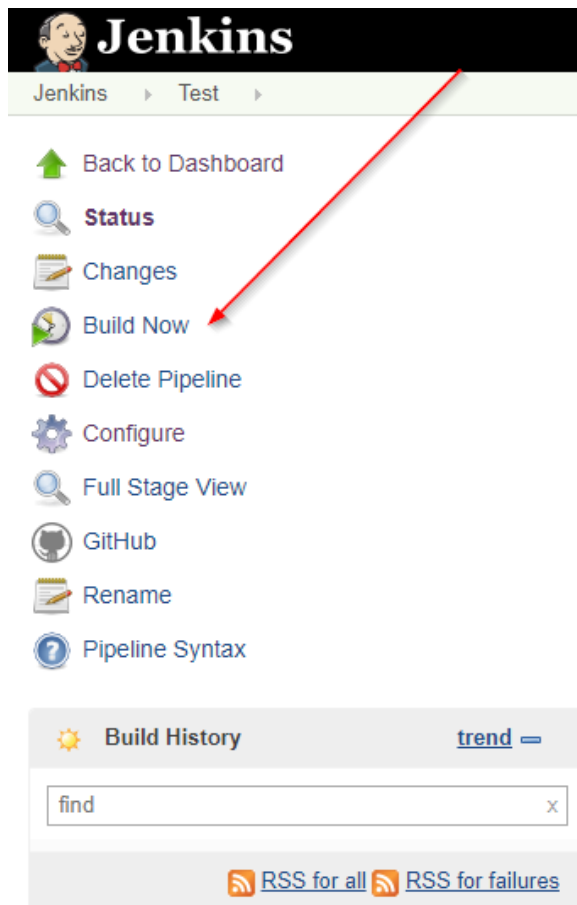
```

As can be seen in the pipeline code above, we have a bunch of steps, first of all we will:

- Define some variables and clone the repository.
- Execute the created tests.
- Build the Spring Boot using the maven plugin, to generate the jar.
- If the build is concluded, it will build the docker image and will stop the current running docker container, running the new one.

Starting the Pipeline

Now we can start the pipeline execution hitting Build Now.



Now the pipeline will start to run, and after some minutes will be concluded. Besides, one important thing to be pointed out, is the fact that this example is related to academic purposes only, note that is not a recommended way of building a pipeline, because we are stopping a docker container, and after this running the new one, besides that we are not managing any cluster, which brings us to 0% of availability when we are applying a patch.